



Documentation Technique Complète - Portfolio

Cette documentation décrit en détail l'architecture, le fonctionnement interne et les choix techniques du projet de Portfolio Web (Laravel / Three.js / Tailwind CSS). Ce document est destiné aux développeurs amenés à maintenir, comprendre ou faire évoluer l'application.



Table des Matières

1. [Architecture Globale](#)
 2. [Modèle de Données \(Base de Données\)](#)
 3. [Routage & Contrôleurs](#)
 4. [L'Écosystème Frontend \(3D, Animations, UI\)](#)
 5. [Gestion des Contenus : Le CMS Hybride](#)
 6. [Système d'Authentification et Sécurité](#)
 7. [Commandes Utiles et Déploiement](#)
-

1. Architecture Globale

Le projet repose sur le framework **Laravel (PHP)** fonctionnant selon le paradigme **MVC (Modèle-Vue-Contrôleur)**.

Stack Technologique

- **Backend** : PHP 8.2+, Laravel (Framework)
- **Frontend Core** : Blade (Moteur de templates), Tailwind CSS (Stylisation utilitaire)
- **Frontend Interactif** : Alpine.js (Comportements réactifs), Lenis (Smooth Scroll), GSAP (Animations avancées)
- **Frontend 3D** : Three.js (Rendu WebGL)
- **Base de données** : MySQL / MariaDB (via Eloquent ORM)
- **Bundler** : Vite (Compilation des assets JS/CSS)

Arborescence Clé (Spécifique au projet)

- `app/Http/Controllers/` : Contient la logique applicative (Publique et `Admin/`).
 - `app/Models/` : Contient les modèles ORM (Project, Article, Experience, AboutPageSetting, User).
 - `resources/views/` : Vues Blade réparties en sous-dossiers (`admin/`, `layouts/`, pages publiques).
 - `resources/js/` : Contient les scripts frontend, avec notamment `three-scene.js` (Logique 3D) et `app.js` (Initialisation Alpine/Lenis).
 - `public/images/projects/` : Dossier très important servant au système de fichiers (File-Based Media).
-

2. Modèle de Données (Base de Données)

L'application utilise l'ORM **Eloquent** pour interagir avec la base de données. Voici la structure des entités principales :

Entité **User**

Gestion de l'administrateur (Accès au Back-Office).

- Champs : **id**, **name**, **email**, **password**, **remember_token**, timestamps.

Entité **Project**

Définition des réalisations (Projets).

- Champs : **id**, **title**, **slug** (URL friendly), **description**, **technologies** (JSON/Array), **github_url**, **demo_url**, timestamps.

Entité **Article**

Gestion d'une éventuelle section blog/veille technologique.

- Champs : **id**, **title**, **slug**, **content**, **published_at**, timestamps.

Entité **AboutPageSetting**

Paramétrage dynamique unique pour la page "À Propos".

- Champs : **id**, **title** (Titre intro), **description** (Texte principal de présentation), **github_link**, **linkedin_link**, **cv_link**, **status** (actif ou inactif).
- Note* : La BDD ne contiendra idéalement qu'une seule ligne active pour cette table, appelée par la vue.

Entité **Experience**

Gestion du parcours (Expériences professionnelles et Formations).

- Champs : **id**, **title** (Nom du poste/diplôme), **company** (Entreprise/École), **location** (Lieu), **start_date** (Date de début), **end_date** (Date de fin, nullable si en cours), **description** (Missions principales), **type** (Enum/String: 'work' ou 'education'), **is_active** (Booléen), timestamps.

3. Routage & Contrôleurs

Le fichier centralisé des routes se trouve dans **routes/web.php**. Le routage est divisé en trois groupes distincts :

Routes Publiques

Accessibles à tout visiteur.

- /** -> **HomeController@index** : Affiche l'Accueil.
- /about** -> **AboutController@index** : Affiche le profil, charge le singleton **AboutPageSetting** et les **Experiences**.
- /projects** & **/projects/{slug}** -> **ProjectController@index/show** : Liste et Détail des projets.
- /articles** & **/articles/{slug}** -> **ArticleController@index/show** : Liste et Détail des articles éducatifs/veille.
- /contact** -> **ContactController@index/send** : Formulaire de contact.

Routes d'Authentification

- `/login` -> `LoginController@showLoginForm` et méthode `login` (POST).
- `/logout` -> (POST) Déconnexion et redirection vers l'accueil.

Routes Administrateur (Back-Office)

Protégées par le middleware `auth`. Préfixées par `/admin`. Gérées principalement par le dossier `App\Http\Controllers\Admin\`.

- `/admin/dashboard` : Tableau de bord principal.
- `/admin/projects/*` : CRUD complet pour les Projets (Création, Édition, Suppression).
- `/admin/images/{id}/delete` : Suppression spécifique de médias associés aux projets.
- `/admin/about` : Édition singleton via `AdminAboutController` (Mise à jour des informations "À Propos").
- `/admin/experiences/*` : CRUD complet via `AdminExperienceController` pour gérer le Parcours.

4. L'Écosystème Frontend (3D, Animations, UI)

L'aspect visuel et sensoriel du site est un point fort du projet.

Scène 3D (`Three.js`)

- **Objectif** : Créer de la profondeur avec un arrière-plan interactif.
- **Fichier responsable** : `resources/js/three-scene.js`.
- **Fonctionnement** :
 1. Instanciation d'un `WebGLRenderer` rattaché à un `<canvas>` en `fixed` z-index négatif.
 2. Création de cibles (Geometries + Textures de Particules) ou réseaux neuronaux ("Points").
 3. Ajout de lumières (`PointLight` aux couleurs cyan/indigo) en orbite mathématique (sin/cos).
 4. **Écouteurs d'événements** : Mise à jour de la caméra sur le `mousemove` (Effet de Parallaxe) et adaptation au `resize`.
 5. Boucle d'animation `requestAnimationFrame`.

Animations Fluides (Lenis & GSAP)

Le scroll par défaut du navigateur est intercepté par **Lenis** pour un défilement inertiel doux ("Smooth Scroll"). **Alpine.js** intercepte la vue (via `x-data` et `x-intersect`) ou via **GSAP** (`ScrollTrigger`) pour animer l'apparition des éléments (`fade-in-up`, `stagger` sur les listes).

Glassmorphism et Tailwind CSS

Les conteneurs de texte pourvoient des classes `bg-gray-800/50` et la propriété `backdrop-blur-md` (flou d'arrière-plan). Ce design assure une lisibilité parfaite des textes blancs par-dessus les éléments 3D filaires en mouvement.

5. Gestion des Contenus : Le CMS Hybride

Ce portfolio est original car il mixe une gestion par Base de Données (Base de données relationnelle) pour la structure, et un système de fichiers brut pour les médias lourds.

Approche Back-Office (BDD)

Les textes descriptifs, le parcours CV (**Experiences**), la configuration du profil de la page About (**AboutPageSetting**), et les descriptions globales des projets sont édités de façon classique depuis le panel Admin (**/admin**). Ces données sont formatées et sécurisées par Eloquent.

Approche File-Based Discovery (Médias & Code des Projets)

Pour afficher une galerie d'image ou des *Snippets de code* complexes sans avoir à gérer des champs BLOB ou l'upload de médias massifs via un formulaire :

1. Le développeur glisse ses assets directement dans **public/images/projects/{slug_du_projet}/**.
2. Le Modèle/Contrôleur du Projet (**ProjectController@show**) **scanne le répertoire public**.
3. Il discrimine les types de fichiers :
 - Fichiers **.jpg, .png, .webp** -> Alimentent le tableau **\$images** injecté dans une galerie Lightbox.
 - Fichiers **.php, .js, .py, .sql** -> Sont lus (**file_get_contents**) et injectés dans un composant "Highlight Code" pour montrer des exploits techniques directement sur la page du projet.

Cette technique est rapide, supprime le besoin de lier les images en base, et gagne un temps précieux sur la publication d'un projet technique.

6. Système d'Authentification et Sécurité

Sécurités en place

1. **Middleware auth** : Obligatoire pour toute route commençant par **/admin**. Refuse l'accès aux utilisateurs non loggés.
2. **Protection CSRF** : Tous les formulaires POST/PUT/DELETE utilisent la directive **@csrf** de Blade pour empêcher les attaques *Cross-Site Request Forgery*.
3. **Validation de Formulaire (Form Requests)** : Toutes les données entrantes (surtout côté AdminExperience et AdminAbout) sont validées stictement (ex: **required|string|max:255**, vérification des dates de fin >= dates de début).
4. **Injection SQL** : Empêchée par l'utilisation exclusive de l'ORM Eloquent ou du Query Builder (mécanismes PDO préparés).
5. **Faible XSS** : Les vues Blade échappent les variables par défaut **{{ \$variable }}**. Les données riches affichées via **{!! \$variable !!}** devront être filtrées si provenant des utilisateurs non admins.

7. Commandes Utiles et Déploiement

Commandes Courantes (Développement)

Pour développer sur le projet localement :

```
# Lancement du serveur Web de développement
php artisan serve

# Lancement de Vite pour la compilation des assets (JS/CSS) en direct
npm run dev

# Nettoyage des caches (si une modification ne s'affiche pas)
php artisan optimize:clear
php artisan route:clear
php artisan view:clear
```

Optimisation pour la Production

Lors du déploiement sur un serveur (ex: VPS, Hébergement Pro), lancez ces commandes pour maximiser les performances de Laravel :

```
# Compilation des assets frontend pour la production
npm run build

# Cache de la configuration (Ne plus modifier le .env sans relancer cette
commande)
php artisan config:cache

# Cache des routes (Extrêmement plus rapide)
php artisan route:cache

# Cache des vues
php artisan view:cache
```

Note de Déploiement

- Assurez-vous que le dossier `storage/` et `bootstrap/cache/` a les droits d'écriture pour l'utilisateur Web (ex: `www-data`).
- Configurez votre serveur Nginx ou Apache de sorte que seul le dossier `public/` soit accessible depuis l'extérieur. Le fichier `.env` doit **toujours resté inatteignable** via le web.

Fin de la documentation.